claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\agent-aa9661dcbb28364ef.jsonl`
- SHA-256: `9477b67427e300c4b55cc53d98521409c866e7ca7115c6a9865c774f9f3327c8`
- Source modified: `2026-06-16T03:13:57+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T03:12:22.688Z)

Map the code that turns a shuttle trajectory into candidate rally windows ? the locus of an "over-merge" problem we just measured (the local detector lumps many rallies + dead time into a few giant windows). cwd = the sports highlight indexer repo.

PRIMARY: Find and read `trajectory_to_action_windows` (in backend/pipeline/detectors/tracknet_runner.py, inherited by the WASB runner) and `parse_trajectory_csv`. Report with file:line anchors:
- The exact windowing algorithm: how "active" frames are determined, how velocity is computed (is it per-frame pixel displacement normalized by frame_width? per-second? fps-aware AT ALL?), how active regions are merged (dead_time_merge_gap) and filtered (min_window_duration).
- The parameters and their defaults/config keys: velocity_threshold (and where it's read ? indexing.<family>.velocity_threshold), dead_time_merge_gap, min_action_window_duration. Is there ANY max_window_duration cap or inactivity-based split? (Confirm absence.)
- The candidate-window dict shape (keys: start, end, peak_velocity, mean_velocity, active_frame_count, track_id_count, chunk_index, ...).
- Explicitly assess: WHY would this over-merge, and WHY would 59.94fps make it worse than ~30fps (velocity scaling)?

SECONDARY: Inventory the OTHER signals available to feed a better segmenter. For each, give file:line + what it computes + per-frame vs per-window + default on/off + how persisted:
- backend/pipeline/detectors/rally_gate.py (Gate-A net_crossings ? the held-shuttle filter)
- backend/pipeline/detectors/person_detector.py (person/court features)
- backend/pipeline/detectors/audio_features.py (audio onset cue)
- backend/eval/fusion_features.py or backend/pipeline/.../fusion_features.py (the ~5 fusion features)
- backend/pipeline/segmenters/fusion_hybrid.py (FusionSegmenter: the _enrich_candidate_stats / _select_for_handoff hooks, Gate-A/B gating)
- the candidate_segments DB table (what stats are persisted per candidate, schema)

Return a precise structured markdown map, anchor-rich, NO full file dumps. Goal: understand exactly how windowing works today (the over-merge mechanism + fps-sensitivity) and what signals already exist to build a better rally segmenter.

2. assistant (2026-06-16T03:12:24.152Z)

I'll systematically map the trajectory-to-windows pipeline and inventory the available signals. Let me start with the core windowing logic.

3. user (2026-06-16T03:12:24.629Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
```

```
          1. Translate the Windows video path to its WSL-visible form (and optionally
             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
          4. Convert that trajectory into high-motion "action windows" ? the same
             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
             pipeline (AI handoff, DB population) is unchanged.

      Nothing here imports TensorFlow; all model code stays in WSL.
      """

      import os
      import csv
      import shlex
      import logging
      import subprocess
      from dataclasses import dataclass
      from typing import List, Dict, Any, Optional, Tuple

      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

      logger = logging.getLogger(__name__)


      @dataclass
      class TrackNetConfig:
          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
          distro: str = "Ubuntu"
          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
          conda_env: str = "TrackNetV4"
          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
          queue_length: int = 5
          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
          keep_frames: bool = False                       # retain staged video after success
(debug only)
          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)

          @classmethod
          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
              tn = (idx_cfg or {}).get("tracknet", {}) or {}
              return cls(
                  distro=tn.get("wsl_distro", "Ubuntu"),
                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                  conda_env=tn.get("conda_env", "TrackNetV4"),
                  weights_path=tn.get("weights_path", ""),
                  queue_length=int(tn.get("queue_length", 5)),
                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                  wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                  timeout_sec=int(tn.get("timeout_sec", 1800)),
                  keep_frames=bool(tn.get("keep_frames", False)),
                  min_free_gb=float(tn.get("min_free_gb", 0.0)),
              )


```

```
    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
```

```
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
```

```
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
```

```
                        y=float(row["Y"]),
                    ))
                except (KeyError, ValueError):
                    continue
        points.sort(key=lambda p: p.frame)
        return points

    @staticmethod
    def trajectory_to_action_windows(
        points: List[TrajectoryPoint],
        fps: float,
        frame_width: float,
        chunk_start: float = 0.0,
        velocity_thresh: float = 0.02,
        merge_gap: float = 2.0,
        min_window_duration: float = 2.0,
        chunk_index: Optional[int] = None,
    ) -> List[Dict[str, Any]]:
        """Convert a shuttle trajectory into candidate rally windows.

        A frame is "active" when the shuttle is visible AND its inter-frame
        displacement (normalised by frame width, per second) exceeds
        ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
        ``merge_gap`` seconds of each other are grouped into one window; short
        windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
        shape feeding the AI-handoff phase is identical.
        """
        if fps <= 0:
            fps = 30.0
        if frame_width <= 0:
            frame_width = 1280.0

        # Compute per-frame normalised velocity from the last *visible* point.
        active = []  # (frame_idx, velocity)
        prev: Optional[TrajectoryPoint] = None
        for p in points:
            if not p.visible:
                prev = None  # break the trajectory; absent shuttle = not in flight
                continue
            if prev is not None:
                dframes = p.frame - prev.frame
                if dframes > 0:
                    dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                    velocity = (dist / frame_width) * (fps / dframes)
                    if velocity > velocity_thresh:
                        active.append((p.frame, velocity))
            prev = p

        if not active:
            return []

        max_gap_frames = int(merge_gap * fps)

        def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
            vels = [v for _, v in group]
            return {
                "start": group[0][0] / fps + chunk_start,
                "end": group[-1][0] / fps + chunk_start,
                "active_frame_count": len(group),
                "peak_velocity": max(vels),
                "mean_velocity": sum(vels) / len(vels),
                "track_id_count": 1,  # single shuttle; kept for window-shape parity
                "chunk_index": chunk_index,
            }

```

```
313             windows = []
314             group = [active[0]]
315             for af in active[1:]:
316                 if af[0] - group[-1][0] <= max_gap_frames:
317                     group.append(af)
318                 else:
319                     windows.append(_finalize(group))
320                     group = [af]
321             windows.append(_finalize(group))
322 
323             return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
324 
```

4. assistant (2026-06-16T03:12:27.012Z)

Good start. Now let me search for where trajectory_to_action_windows is actually called (parameters source) and look at the other detector signals.

5. user (2026-06-16T03:12:27.507Z)

```
Found 16 files
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\eval\windowing.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\rally_gate.py
backend\eval\calibrate_local.py
backend\eval\rally_seq_proto.py
backend\eval\eval_partial_labels.py
backend\eval\calibrate_wasb.py
backend\eval\export_wasb_segments.py
```

6. user (2026-06-16T03:12:27.643Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2 
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8 
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15 
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21 
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24 
```

```
    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]

```

```
90
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94          """Annotate each (start,end) window with its shuttle net-crossing count and a
95          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96          axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98          `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99          `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                    kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118
```

7. user (2026-06-16T03:12:28.062Z)

```
1       """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3       Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7       later the learned classifier reads its persisted features. Keeping detection here (not
in
8       the segmenter) is what makes "add a cue = register a producer" true.
9
10      Behaviour is unchanged from the in-lined version ? only the home moved:
11        - `lazy_load_model`              ? load a permissively-licensed torchvision detector
12        - `make_tracker`                 ? a fresh supervision ByteTrack per chunk
13        - `detect_and_track`             ? one frame ? confident persons with persistent IDs
14        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16          that feed the candidate `stats` JSON and the learned fusion features.
17
18      The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19      detector weights load only on first detection (`lazy_load_model`) ? so importing the
20      segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21      are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22      machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23      (ADR-005; ultralytics' AGPL YOLO was dropped).
24      """
25      import logging
26      from typing import Any, Callable, Dict, List, Optional, Tuple
27
```

```
    import cv2
    import torch

    from backend.utils.geometry import get_velocity_normalised

    logger = logging.getLogger(__name__)

    # torchvision detection models are trained on COCO where the "person" category is
    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
    _PERSON_CLASS_ID = 1

    # Permissively-licensed torchvision detectors selectable via config
    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
    _DETECTOR_FACTORIES = {
        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
    }
    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"


    class PersonDetector:
        """Stage-1 person detection + tracking + motion-window extraction (a rally cue).

        Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
        ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
        min_action_window_duration}``); the model loads lazily on first use.
        """

        def __init__(self, config: Dict[str, Any]):
            self.config = config
            # Dynamic torchvision module (None until lazy_load_model); typed Any so the
            # `self.model([tensor])` inference call type-checks without a quarantine.
            self.model: Any = None
            self.device: Optional[str] = None

        def lazy_load_model(self) -> None:
            if self.model is not None:
                return

            # Imported lazily so the module imports without torchvision present and so
            # the test suite (which pre-sets self.model) never pulls in model weights.
            from torchvision.models import detection as tv_detection

            idx_cfg = self.config.get("indexing", {})
            use_gpu = idx_cfg.get("use_gpu", True)
            self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
            if use_gpu and not torch.cuda.is_available():
                logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")

            model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
            if model_name not in _DETECTOR_FACTORIES:
                logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
                model_name = _DEFAULT_DETECTOR

            builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
            logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
            # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
            self.model = builder(weights="DEFAULT")
            self.model.eval()
            self.model.to(self.device)

```

```
91          def make_tracker(self, fps: float) -> Any:
92              """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94              Imported lazily so tests can mock this method without supervision installed.
95              """
96              import supervision as sv
97              frame_rate = max(1, int(round(fps)))
98              return sv.ByteTrack(frame_rate=frame_rate)
99
100         def detect_and_track(self, frame: Any, tracker: Any,
```

8. user (2026-06-16T03:12:28.274Z)

```
1       """Pure per-window audio-cue features for rally fusion (fusion P4).
2
3       NO I/O ? the audio hit detection lives in `backend/pipeline/audio/hit_detector.py`
4       (`detect_hits_in_video` -> `List[HitEvent]`); this only turns those hit events into a
few
5       per-window features the learned scorer can weight. Mirrors `fusion_features.py`
discipline.
6
7       Honest expectation (see the `audio-e1-findings` memory): the classical-onset audio cue
had
8       only ~2.2x discrimination and was NOT adopted standalone ? so this is "add the signal as
a
9       column and let the golden-set harness decide", default-OFF, not "adopt audio".
10      """
11      from __future__ import annotations
12
13      from typing import Dict, List
14
15      AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]
16
17
18      def compute_audio_features(hits: List, window_start: float, window_end: float) ->
Dict[str, float]:
19          """Audio features for one window from a video's HitEvents (each has ``.t``,
``.strength``).
20
21          A real rally is a sustained volley of racket?shuttle contacts; dead time / a single
22          service feed has few. Returns hit count, hits/second, and mean strength ? all 0.0 on
an
23          empty window. ``[start, end)`` half-open so a hit on a boundary belongs to one
window only.
24          """
25          dur = max(1e-6, window_end - window_start)
26          in_win = [h for h in (hits or []) if window_start <= h.t < window_end]
27          if not in_win:
28              return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
29          n = len(in_win)
30          return {
31              "audio_hit_count": float(n),
32              "audio_hit_rate": n / dur,
33              "audio_hit_strength_mean": sum(h.strength for h in in_win) / n,
34          }
35
```

9. assistant (2026-06-16T03:12:29.920Z)

Now let me find the fusion_features and FusionSegmenter files:

10. user (2026-06-16T03:12:30.245Z)

backend\pipeline\detectors\fusion_features.py

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\local-noai\backend\pipeline\detectors\fusion_features.py
```

11. user (2026-06-16T03:12:30.449Z)

```
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\local-noai\backend\pipeline\segmenters\fusion_hybrid.py
```

12. user (2026-06-16T03:12:30.991Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/eval/fusion_audio.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/eval/fusion_compare.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/eval/fusion_golden.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend/pipeline/detectors/fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/fusion_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/motion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend/pipeline/segmenters/tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend/pipeline/segmenters/trajectory_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/wasb_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/yolo_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/pipeline/segmenters/__init__.py
```

13. assistant (2026-06-16T03:12:32.359Z)

Now let me read the key files:

14. user (2026-06-16T03:12:32.799Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
```

```
    from __future__ import annotations

    from typing import Dict, List, Optional, Sequence, Tuple

    BBox = Tuple[float, float, float, float]
    Net = Tuple[str, float]


    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
        if polygon is None or len(polygon) < 3:
            return False
        x, y = point
        inside = False
        n = len(polygon)
        j = n - 1
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
        point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
        if court_polygon is None:
            return True
        if frame_width <= 0 or frame_height <= 0:
            return False
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
```

```
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, with x
        normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
        not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
```

```
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
            aligned += 1
            if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
                   for _tid, box in dets):
                colocated += 1
        return colocated / aligned if aligned else 0.0


    # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
    # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
    PERSON_FEATURE_NAMES = [
        "players_in_court", "two_sided_motion", "mean_player_motion",
        "in_court_ratio", "shuttle_player_colocation",
    ]


    def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                frame_width: float, frame_height: float,
                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
                                net: Optional[Net] = None,
```

```
                                 min_players: int = 2) -> Dict[str, float]:
        """Aggregate the per-window person features from per-frame detections + the in-
window
        shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
        counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
        return {
            "players_in_court": players_in_court(counts),
            "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
            "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
            "in_court_ratio": in_court_ratio(counts, min_players),
            "shuttle_player_colocation": shuttle_player_colocation(
                per_frame, shuttle_points, frame_width, frame_height),
        }

```

15. user (2026-06-16T03:12:32.979Z)

```
      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
      shuttle-seeded window via two overridable seams the engine exposes:

      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
        GPU-free, computed from the trajectory we already have); and, only when the person cue
        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
        **Persisted candidates remain the full superset** regardless, so the offline
experiment
        harness can re-score any variant.

      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
      experiment harness (EXPERIMENT_HARNESS.md), never silently.
      """
      from typing import Any, Dict, List, Optional, Tuple

      from backend.pipeline.segmenters.base import segmenter_registry
      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
      from backend.pipeline.detectors.base import DetectorRunner
      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
      from backend.pipeline.detectors import rally_gate
      from backend.pipeline.detectors import fusion_features as ff

```

```
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
```

```
fcfg))
            return stats

        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
```

```
139                                             court_polygon=polygon, net=net)
140
141         def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                                 window_stats: List[Dict[str, Any]],
143                                 idx_cfg: Dict[str, Any]) -> List[int]:
144             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145             (0) keep every window ? identical to the substrate. Persisted candidates are
146             untouched (full superset)."""
147             fcfg = idx_cfg.get("fusion", {})
148             min_crossings = int(fcfg.get("min_crossings", 0))
149             min_players = int(fcfg.get("min_players", 0))
150             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
151
152             keep: List[int] = []
153             for i, st in enumerate(window_stats):
154                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
155                     continue
156                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
157                     continue
158                 keep.append(i)
159             return keep
160
161
162     segmenter_registry.register(FusionSegmenter)
163
```

16. assistant (2026-06-16T03:12:38.229Z)

Now let me check where trajectory_to_action_windows is invoked and what parameters are used. Let me also get the DB schema:

17. user (2026-06-16T03:12:39.181Z)

```
backend/pipeline/detectors/base.py:33:   TrackNetRunner.parse_trajectory_csv and
.trajectory_to_action_windows (they are
backend/pipeline/detectors/base.py:50:- Write the exact same CSV format so
trajectory_to_action_windows works verbatim.
backend/pipeline/detectors/base.py:191:    trajectory_to_action_windows: Callable[..., Any]
backend/pipeline/detectors/rally_gate.py:18:windows from trajectory_to_action_windows, or to
fold into it later.
backend/pipeline/detectors/tracknet_runner.py:256:    def trajectory_to_action_windows(
backend/pipeline/detectors/wasb_runner.py:20:# parse_trajectory_csv /
trajectory_to_action_windows are @staticmethods on TrackNetRunner.
backend/pipeline/detectors/wasb_runner.py:199:    trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
backend/pipeline/segmenters/trajectory_hybrid.py:197:        all_candidate_windows =
runner.trajectory_to_action_windows(
```

18. user (2026-06-16T03:12:39.973Z)

```
backend\config\models.py:43:    velocity_threshold: float = 0.02
backend\config\models.py:69:    velocity_threshold: float = 0.02
backend\config\models.py:97:    # indexing.<family>.velocity_threshold). Default matches the
substrates' 0.02; set
backend\config\models.py:98:    # equal to indexing.wasb.velocity_threshold to A/B fusion
against a tuned wasb run.
backend\config\models.py:99:    velocity_threshold: float = 0.02
backend\config\models.py:122:    dead_time_merge_gap: float = 2.0
backend\config\models.py:127:    yolo_velocity_threshold: float = 0.15
backend\config\models.py:131:    min_action_window_duration: float = 2.0
backend\eval\calibrate_local.py:6:    WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)
```

```
backend\eval\calibrate_local.py:34:# (velocity_thresh, merge_gap, min_window_duration,
min_crossings)
backend\eval\calibrate_local.py:48:            velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\pipeline\detectors\person_detector.py:55:    min_action_window_duration}``); the model
loads lazily on first use.
backend\pipeline\detectors\person_detector.py:141:
velocity_thresh: float, merge_gap: float,
backend\pipeline\detectors\person_detector.py:206:                    if velocity >
velocity_thresh:
backend\pipeline\detectors\person_detector.py:227:        max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\person_detector.py:253:        min_window_duration =
self.config.get("indexing", {}).get("min_action_window_duration", 2.0)
backend\pipeline\detectors\person_detector.py:254:        windows = [w for w in windows if
(w["end"] - w["start"]) >= min_window_duration]
backend\eval\calibrate_wasb.py:28:Config = Tuple[float, float, float]   # (velocity_thresh,
merge_gap, min_window_duration)
backend\eval\calibrate_wasb.py:52:            velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\eval\distill_local.py:43:# candidates (the classifier removes the junk).
(velocity_thresh, merge_gap, min_dur).
backend\eval\distill_local.py:59:        velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:81:        velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\pipeline\segmenters\motion.py:44:        dead_time_merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\motion.py:141:        # Post-Processing: 1. Merge segments separated
by less than dead_time_merge_gap
backend\pipeline\segmenters\motion.py:146:                if next_start - curr_end <
dead_time_merge_gap:
backend\pipeline\detectors\tracknet_runner.py:261:        velocity_thresh: float = 0.02,
backend\pipeline\detectors\tracknet_runner.py:262:        merge_gap: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:263:        min_window_duration: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:270:        ``velocity_thresh`` ? i.e. the shuttle
is in flight. Active frames within
backend\pipeline\detectors\tracknet_runner.py:271:        ``merge_gap`` seconds of each other
are grouped into one window; short
backend\pipeline\detectors\tracknet_runner.py:292:                    if velocity >
velocity_thresh:
backend\pipeline\detectors\tracknet_runner.py:299:        max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\tracknet_runner.py:323:        return [w for w in windows if
(w["end"] - w["start"]) >= min_window_duration]
backend\eval\eval_partial_labels.py:132:# min_window_duration DOWN (recover short 3-5s rallies)
and merge_gap UP (heal
backend\eval\eval_partial_labels.py:146:    """Grid-search (velocity, merge_gap, min_dur) x gate
vs human labels, in-window.
backend\eval\eval_partial_labels.py:186:    print(f"\nbest: velocity={b['cfg'][0]}
merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]} "
backend\pipeline\segmenters\trajectory_hybrid.py:47:    #: config section under `indexing`
(velocity_threshold lives there), the
backend\pipeline\segmenters\trajectory_hybrid.py:160:        velocity_thresh =
idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
backend\pipeline\segmenters\trajectory_hybrid.py:161:        merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\trajectory_hybrid.py:162:        min_window_duration =
idx_cfg.get("min_action_window_duration", 2.0)
backend\pipeline\segmenters\trajectory_hybrid.py:170:            "velocity_thresh":
velocity_thresh,
backend\pipeline\segmenters\trajectory_hybrid.py:171:            "merge_gap": merge_gap,
backend\pipeline\segmenters\trajectory_hybrid.py:172:            "min_action_window_duration":
min_window_duration,
backend\pipeline\segmenters\trajectory_hybrid.py:199:
velocity_thresh=velocity_thresh, merge_gap=merge_gap,
backend\pipeline\segmenters\trajectory_hybrid.py:200:
```

```
min_window_duration=min_window_duration, chunk_index=0,
backend\eval\export_wasb_segments.py:35:# The tuned config from the first per-video calibration
(velocity, merge_gap, min_dur);
backend\eval\export_wasb_segments.py:219:    cfg = (args.velocity_thresh, args.merge_gap,
args.min_duration)
backend\eval\experiment.py:103:      - ``threshold`` ? classifier probability cutoff.
``merge_gap`` ? post-filter
backend\eval\experiment.py:116:    merge_gap: float = 1.0
backend\eval\experiment.py:138:            "merge_gap": self.merge_gap,

[Showing results with pagination = limit: 50]
```

19. assistant (2026-06-16T03:12:41.581Z)

Good! Now let me check the trajectory_hybrid segmenter to see the full call context:

20. user (2026-06-16T03:12:42.220Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
```

```
        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

```

```
106         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
107                           player_pool: Optional[List[str]] = None,
108                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109             # override-ignore: the ABC declares the Result contract (Success|Failure)
110             # but every live segmenter still returns a bare list ? the documented
111             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112             label = self.display_label
113             # --- Sport profile + AI provider wiring (identical across segmenters) ---
114             sport_name = self.config.get("sport", "badminton")
115             try:
116                 sport_profile = sport_registry.get(sport_name)
117             except KeyError as e:
118                 logger.error(str(e))
119                 return []
120 
121             idx_cfg = self.config.get("indexing", {})
122             # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123             # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124             # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125             # measure it against the Gemini path, or to avoid the cloud entirely). With
126             # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127             skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129             if skip_ai:
130                 model_name = "local-noai"
131                 logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                             "be emitted as rallies; no %s handoff, no API key needed.",
133                             label, provider_name)
134             else:
135                 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
137                 api_key = os.environ.get(api_key_env_var)
138                 if not api_key:
139                     logger.error(
140                         f"{api_key_env_var} environment variable is not set! "
141                         f"To run the {provider_name} handoff, set your API key. "
142                         f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                     )
144                     return []
145                 try:
146                     provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147                 except KeyError as e:
148                     logger.error(str(e))
149                     return []
150 
151             video_duration = get_video_duration(video_path)
152             if video_duration <= 0:
153                 logger.error("Invalid video duration.")
154                 return []
155             fps, frame_width = self._probe_fps_width(video_path)
156 
157             # --- Runner config + windowing thresholds ---
158             runner, runner_params = self._build_runner(idx_cfg)
159 
160             velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162             min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
```

```
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
```

```
222                 for i, cw in enumerate(all_candidate_windows):
223                     candidate_ids[i] = self.db.add_candidate_segment(
224                         video_id, detector=self.name,
225                         start_time=cw["start"], end_time=cw["end"],
226                         chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
227                         stats=window_stats[i], detection_params=detection_params,
228                     )
229
230             if not all_candidate_windows:
231                 return []
232
233             # --- Phase 3: AI provider handoff over padded candidate clips ---
234             # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
235             # candidates above stay the full superset ? only the served play_segments are
gated.
236             handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
237
238             ai_segments: List[dict] = []
239             if skip_ai:
240                 # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
241                 # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
242                 # falls back to the duration-based estimate (no AI exchange count).
243                 ai_segments = [
244                     {
245                         "start_time": all_candidate_windows[wi]["start"],
246                         "end_time": all_candidate_windows[wi]["end"],
247                         "shuttle_exchange_count": None,
248                         "shot_count_confidence": "LocalNoAI",
249                         "_candidate_id": candidate_ids[wi],
250                     }
```

21. user (2026-06-16T03:12:43.323Z)

```
(Bash completed with no output)
```

22. assistant (2026-06-16T03:12:46.064Z)

Let me check the config models and the DB schema:

23. user (2026-06-16T03:12:46.508Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
```

```
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
        # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
        min_free_gb: float = 0.0
        # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
        # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
```

```
        # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
        # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
        # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
        stream_video: bool = False


    class FusionConfig(BaseModel):
        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).

        Every default reproduces control behaviour: the fusion segmenter persists the
        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
        true ? the person-cue features, but it does NOT gate the served handoff unless a
        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
        supplied ? off-court persons (spectators / adjacent courts) are excluded.
        """
        # Which trajectory substrate seeds the windows (shuttle stays primary).
        substrate: Literal["wasb", "tracknet"] = "wasb"
        # Windowing velocity threshold for the fusion family (the engine reads
        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
        velocity_threshold: float = 0.02
        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
        # enabled ? so the default path never imports torch / touches the GPU.
        cue_flags: dict[str, bool] = Field(default_factory=dict)
        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
        min_crossings: int = Field(default=0, ge=0)
        # Served Gate B: when the person cue is on, drop windows with median in-court
players
        # < this. 0 = OFF.
        min_players: int = Field(default=0, ge=0)
        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
        court_polygon: Optional[list[list[float]]] = None
        # Net line for the person two-sided-motion split (person features only ? the shuttle
        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
        net_axis: Literal["x", "y"] = "y"
        net_position: float = Field(default=0.5, ge=0.0, le=1.0)


    class IndexingConfig(BaseModel):
        default_segmenter: str = "yolo_hybrid"
        use_gpu: bool = True
        motion_threshold: float = 0.08
        min_segment_duration: float = 3.0
        dead_time_merge_gap: float = 2.0
        chunk_duration: float = 90.0
        chunk_overlap: float = 10.0
        detector_model: str = "fasterrcnn_resnet50_fpn"
        detector_score_threshold: float = 0.5
        yolo_velocity_threshold: float = 0.15
        yolo_frame_skip: int = 3
        yolo_tracker: str = "bytetrack.yaml"
        yolo_prefetch_workers: int = 2
        min_action_window_duration: float = 2.0
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
```

```
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
        # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
```

```
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
```

```
252         security: SecurityConfig = Field(default_factory=SecurityConfig)
253         logging: LoggingConfig = Field(default_factory=LoggingConfig)
254
255         # Allow extra keys during transition so existing config.json files do not break.
256         model_config = {
257             "extra": "allow",
258             "validate_assignment": True,
259         }
260
261         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
262         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
263         # attribute access / the typed config. A nested *section* is returned as a plain
dict
264         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
265         # section model instead breaks it (a model has no .get), which bit every validator
on
266         # real runs. Scalar leaves are returned as-is.
267         def get(self, key: str, default: Any = None) -> Any:
268             _MISSING = object()
269             val: Any = _MISSING
270             if key in type(self).model_fields or hasattr(self, key):
271                 val = getattr(self, key, _MISSING)
272             if val is _MISSING:
273                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
274                 # discovered dynamically from the model fields, so a future top-level
section
275                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
276                 for fname in type(self).model_fields:
277                     section = getattr(self, fname, None)
278                     if isinstance(section, BaseModel) and hasattr(section, key):
279                         val = getattr(section, key)
280                         break
281             if val is _MISSING:
282                 return default
283             if isinstance(val, BaseModel):
284                 return val.model_dump(mode="json")
285             return val
286
287         def __getitem__(self, key: str) -> Any:
288             val = self.get(key)
289             if val is None and key not in type(self).model_fields:
290                 # Re-raise like a real dict for unknown top-level
291                 raise KeyError(key)
292             return val
293
294
295     # Convenience type alias for callers that want to annotate
296     Config = AppConfig
297
```

24. user (2026-06-16T03:12:47.057Z)

(Bash completed with no output)

25. assistant (2026-06-16T03:12:48.714Z)

Now let me find the DB schema and candidate_segments table:

26. user (2026-06-16T03:12:52.001Z)

./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_database.py
./.claude/worktrees/bhi-execpolicy/backend/infrastructure/database.py
./.claude/worktrees/bhi-execpolicy/tests/test_database.py
./.claude/worktrees/local-noai/backend/infrastructure/database.py
```

27. user (2026-06-16T03:12:52.309Z)

```
(Bash completed with no output)
```

28. user (2026-06-16T03:12:54.210Z)

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/execution_policy.py
./.claude/worktrees/bhi-execpolicy/backend/infrastructure/database.py
./.claude/worktrees/bhi-execpolicy/backend/infrastructure/execution_policy.py
./.claude/worktrees/local-noai/backend/infrastructure/database.py
./.claude/worktrees/local-noai/backend/infrastructure/execution_policy.py
./backend/infrastructure/database.py
./backend/infrastructure/execution_policy.py
```

29. user (2026-06-16T03:12:55.867Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
29          def _get_connection(self):
30              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32              conn = sqlite3.connect(self.db_path, timeout=30.0)
33              conn.row_factory = sqlite3.Row
34              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35              conn.execute("PRAGMA foreign_keys = ON")
36              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37              conn.execute("PRAGMA busy_timeout = 30000")
38              try:
39                  conn.execute("PRAGMA journal_mode = WAL")
40              except sqlite3.OperationalError:
41                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
```

```
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
```

```
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
```

```
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")

                if version < 4:
                    # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
                    # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
                    # path can scope rows to an owner. **NULL = local install = byte-
identical to
                    # today** ? every existing row stays NULL and every existing query
(which never
                    # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
                    # Additive ONLY: no served-behavior change rides on this slice.
                    cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
                    cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
                    # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
                    # of index churn while making the eventual per-user lookups cheap.
                    cursor.execute(
                        "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
                        "WHERE user_id IS NOT NULL")
                    cursor.execute(
                        "CREATE INDEX IF NOT EXISTS idx_candidates_user "
                        "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
                    cursor.execute("PRAGMA user_version = 4")

                if version < 5:
                    # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
                    # version): the resolved per-user `fusion_config` overlay + an INLINE
                    # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
                    # and provenance that earned it. `is_active` pins the one row the
serving bridge
                    # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
                    # only persists/loads it ? NOTHING reads it on the served path yet.
                    #
                    #   user_id              ? owner of the model (NULL allowed = the local
install)
                    #   version              ? monotonic per-user model version
                    #   baseline_version     ? the shared baseline this was fit against
(upgrade switch)
                    #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
                    #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
                    #   classifier_json      ? JSON LogisticRegression.to_dict() (None =
config-only model)
                    #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it
was promoted)
                    #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n
trust floor input)
                    #   is_active            ? the one resolved row for this user (partial-
unique below)
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS user_models (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            user_id TEXT,
                            version INTEGER NOT NULL DEFAULT 1,
                            baseline_version TEXT,
                            feature_schema_hash TEXT,
                            fusion_config TEXT,
```

```
                            classifier_json TEXT,
                            promotion_evidence TEXT,
                            n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
                            is_active INTEGER NOT NULL DEFAULT 0,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                        )
                    """)
                    # One model version per user (NULLs compare distinct in SQLite, so the
local
                    # install can still carry multiple versioned rows; the active-row guard
below
                    # is what enforces a single served model).
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
                            ON user_models(user_id, version)
                    """)
                    # At most ONE active model per user: a partial-unique index over the
active rows.
                    # (SQLite treats NULL user_id rows as distinct, so the local install's
single
                    # active row is still guarded ? there is one local user.)
                    cursor.execute("""
                        CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
                            ON user_models(user_id) WHERE is_active = 1
                    """)
                    cursor.execute("PRAGMA user_version = 5")
                # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

```

```
259             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
260             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
261             """
262             with self._connect() as conn:
263                 cur = conn.cursor()
264                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
265                                 (video_id,)).fetchone()[0]
266                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
267                                 (video_id,)).fetchone()[0]
268                 return int(p), int(c)
269
270         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
271                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
272                            cand_after: Optional[int] = None) -> None:
273             """Delete play/candidate segments by id range (events cascade from
play_segments).
274
275             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
276             successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
277             and keep the OLD ones when the run failed/produced nothing (``*_after``).
278             """
279             with self._connect() as conn:
280                 cur = conn.cursor()
281                 if play_upto is not None:
282                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
283                 if play_after is not None:
284                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
285                 if cand_upto is not None:
286                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
287                 if cand_after is not None:
288                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
289                 conn.commit()
290
291         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292             try:
293                 with self._connect() as conn:
294                     cursor = conn.cursor()
295                     cursor.execute(
296                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
298                     )
299                     conn.commit()
300                     return cursor.lastrowid
```

30. assistant (2026-06-16T03:12:58.142Z)

Let me continue reading the DB schema to see the add_candidate_segment method:

31. user (2026-06-16T03:12:58.791Z)

```
300                     return cursor.lastrowid
301             except Exception as e:
302                 logger.error(f"Failed to add play segment: {e}")
```

```
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
```

```
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
```

```
419                 cursor = conn.cursor()
420                 rows = cursor.execute(query, params).fetchall()
421
422                 for row in rows:
423                     r_dict = dict(row)
424                     r_id = r_dict["id"]
425                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427                     # Fetch associated events
428                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429                     r_dict["events"] = []
430                     for er in event_rows:
431                         e_dict = dict(er)
432                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
433                         r_dict["events"].append(e_dict)
434
435                     segments_list.append(r_dict)
436
437             return segments_list
438
439         def clear_video_data(self, video_id: str):
440             with self._connect() as conn:
441                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442                 conn.commit()
443
444         # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
445         # Writers follow the repo convention (swallow + logger.error, return bool); the
446         # worker logs loudly on a failed transition so a job can't silently wedge.
447
448         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                        player_pool: Optional[List[str]] = None,
```

32. user (2026-06-16T03:12:59.244Z)

```
(Bash completed with no output)
```

33. assistant (2026-06-16T03:13:00.950Z)

Now let me check person_detector to understand its windowing and feature computation:

34. user (2026-06-16T03:13:01.594Z)

```
100         def detect_and_track(self, frame: Any, tracker: Any,
101                              score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102             """Detect persons in a frame and assign persistent track IDs.
103
104             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105             This replaces the old ultralytics ``model.track(...)`` call, which bundled
106             detection AND association: torchvision does the detection, supervision's
107             ByteTrack does the association.
108             """
109             import supervision as sv
110
111             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113             tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114             with torch.no_grad():
115                 outputs = self.model([tensor])
116
117             out = outputs[0]
118             boxes = out["boxes"].detach().cpu().numpy()
```

```
            labels = out["labels"].detach().cpu().numpy()
            scores = out["scores"].detach().cpu().numpy()

            # Keep only confident person detections.
            mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
            if not mask.any():
                return []

            # ByteTrack requires confidence to be populated (it filters on it internally).
            dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
            tracked = tracker.update_with_detections(dets)

            result: List[Tuple[int, Tuple[float, float, float, float]]] = []
            for i in range(len(tracked)):
                tid = tracked.tracker_id[i]
                if tid is None:
                    continue
                x1, y1, x2, y2 = tracked.xyxy[i]
                result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
            return result

        def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
                                              velocity_thresh: float, merge_gap: float,
                                              frame_skip: int = 3,
                                              chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
            """
            Runs person detection + tracking on a video chunk and extracts high-motion
            action windows. Returns a list of dicts in the full video's timeframe, each
with:
                start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
                track_id_count, chunk_index.

            Args:
                frame_skip: Process every Nth frame (1 = every frame). Higher values
                            dramatically reduce inference cost with minimal quality loss.
                chunk_index: Index of the source chunk (recorded on each window for
traceability).
            """
            cap = cv2.VideoCapture(chunk_path)
            if not cap.isOpened():
                logger.error(f"Could not open {chunk_path}")
                return []

            fps = cap.get(cv2.CAP_PROP_FPS)
            if fps <= 0:
                fps = 30.0

            frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
            if frame_width <= 0:
                frame_width = 1280.0  # Sensible fallback

            # Effective sampling rate after frame-skipping
            effective_fps = fps / max(frame_skip, 1)

            # A fresh tracker per chunk ? track IDs only need to be consistent within a
            # chunk (windows are computed per chunk, then merged across chunks by time).
            tracker = self.make_tracker(effective_fps)
            score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)

            # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
            # so windows can be enriched with motion stats for logging + fine-tuning.
            active_frames = []
```

```
            frame_idx = 0
            track_history: Dict[int, Tuple[float, float, float, float]] = {}

            while True:
                ret, frame = cap.read()
                if not ret:
                    break

                # Skip frames for performance ? at 30fps, every 3rd frame still
                # gives ~10 samples/second, plenty for human-scale velocity.
                if frame_idx % max(frame_skip, 1) != 0:
                    frame_idx += 1
                    continue

                # Detect persons + assign persistent track IDs.
                detections = self.detect_and_track(frame, tracker, score_thresh)

                frame_peak_velocity = 0.0
                frame_active_ids = set()
                current_ids = set()

                for track_id, bbox in detections:
                    current_ids.add(track_id)

                    if track_id in track_history:
                        prev_bbox = track_history[track_id]
                        velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
                        if velocity > velocity_thresh:
                            frame_peak_velocity = max(frame_peak_velocity, velocity)
                            frame_active_ids.add(track_id)

                    track_history[track_id] = bbox

                # Prune stale IDs that are no longer visible ? prevents false
                # velocity spikes when the tracker reassigns an old ID.
                track_history = {k: v for k, v in track_history.items() if k in current_ids}

                if frame_active_ids:
                    active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))

                frame_idx += 1

            cap.release()

            # Group active frames into continuous windows
            if not active_frames:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
                velocities = [v for _, v, _ in group]
                track_ids = set().union(*[ids for _, _, ids in group])
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(velocities),
                    "mean_velocity": sum(velocities) / len(velocities),
                    "track_id_count": len(track_ids),
                    "chunk_index": chunk_index,
                }

            windows = []
```

```
            group = [active_frames[0]]
            for af in active_frames[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            # Filter out extremely short action windows
            min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
            windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

            return windows

        def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
                                 frame_skip: int = 3) -> Dict[str, Any]:
            """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.

            Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
            ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
discards the
            per-frame boxes), this returns the raw per-frame detections so the fusion
features can
            align them with the shuttle trajectory. It reads the source by **absolute frame
index**
            (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
            the WASB trajectory's ``.frame`` exactly ? that alignment is what
            ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.

            Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
            "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
            accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
            treat the indices as ±a few frames on real footage ? fine for window-level
features.
            """
            self.lazy_load_model()
            cap = cv2.VideoCapture(video_path)
            if not cap.isOpened():
                logger.error(f"Could not open {video_path}")
                return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}

            fps = cap.get(cv2.CAP_PROP_FPS)
            if fps <= 0:
                fps = 30.0
            frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
            frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
            score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
            step = max(frame_skip, 1)
            tracker = self.make_tracker(fps / step)

            start_frame = max(0, int(start_frame))
            frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
            cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
            frame_idx = start_frame
            while frame_idx <= end_frame:
                ret, frame = cap.read()
                if not ret:
                    break
```

```
298                 if (frame_idx - start_frame) % step == 0:
299                     frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
```

35. assistant (2026-06-16T03:13:03.437Z)

Now let me check the wasb_runner to see the full implementation there:

36. user (2026-06-16T03:13:03.949Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43          # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44          # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45          # extraction that dominates a long clip. Output is bit-identical to the PNG path
46          # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47          stream_video: bool = False
48
49          @classmethod
```

```
50          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51              w = (idx_cfg or {}).get("wasb", {}) or {}
52              return cls(
53                  distro=w.get("wsl_distro", "Ubuntu"),
54                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56                  conda_env=w.get("conda_env", "wasb"),
57                  weights_path=w.get("weights_path",
58                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59                  sport=w.get("sport", "badminton"),
60                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61                  timeout_sec=int(w.get("timeout_sec", 1800)),
62                  keep_frames=bool(w.get("keep_frames", False)),
63                  min_free_gb=float(w.get("min_free_gb", 0.0)),
64                  stream_video=bool(w.get("stream_video", False)),
65              )
66
67
68      class WasbRunner(WslCommandMixin, DetectorRunner):
69          def __init__(self, cfg: WasbConfig):
70              self.cfg = cfg
71
72          def healthcheck(self) -> Tuple[bool, str]:
73              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74              cmd = (f"conda activate {self.cfg.conda_env} && "
75                     f"python -c \"import torch; print('torch', torch.__version__, "
76                     f"'cuda', torch.cuda.is_available())\"")
77              try:
78                  res = self._wsl_bash(cmd)
79              except FileNotFoundError:
80                  return False, "`wsl` not found ? Windows + WSL2 required."
81              except subprocess.TimeoutExpired:
82                  return False, "WSL healthcheck timed out."
83              out = (res.stdout or "").strip()
84              if res.returncode != 0:
85                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86              return True, out
87
88          def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89              src_mnt = to_wsl_mnt_path(video_win_path)
90              reason = self.wsl_source_unreadable_reason(src_mnt)
91              if reason:
92                  logger.error("Cannot stage video into WSL: %s", reason)
93                  return None
94              dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95              cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96              try:
97                  res = self._wsl_bash(cmd)
98              except subprocess.TimeoutExpired:
99                  logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)
```

```
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.

            All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
            two videos that share a filename (e.g. two ``match.mp4`` from different folders)
            can never reuse each other's staged frames, resumable cache, or CSV.
            """
            # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
            # translation: to_wsl_mnt_path passes relative paths through unchanged, so
            # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
            # the CSV landed inside WSL while Windows polled the repo-relative path.
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize for cross-platform basename (tests use Windows paths on Linux).
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)
            # Unique, content-derived key: same filename + different bytes -> different key.
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            wsl_out_dir = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")

            if not self._stage_infer_script():
                logger.error("Could not stage wasb_infer.py into WSL.")
                return None
            staged_video = self._stage_video(video_win_path, f"{key}{ext}")
            if staged_video is None:
                return None
            frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"

            # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
            # WSL stage filesystem is already low so the user can clean up before it fills.
            # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
            if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning(
                        "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
                        "may fill the disk ? consider running tools/cleanup_caches.py.",
                        free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            # Fast path: stream-decode the video in-process (no PNG extraction). Output is
            # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
            if self.cfg.stream_video:
                frame_args = "--stream-video "
            else:
                frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
            cmd = (
                f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
                f"python wasb_infer.py "
                f"--video {wsl_tilde_quote(staged_video)} "
                f"{frame_args}"
                f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--sport {shlex.quote(self.cfg.sport)} "
                f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
            )
            logger.info("Running WASB inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
```

```
173             except subprocess.TimeoutExpired:
174                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175                 return None
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178                 return None
179             if not os.path.exists(expected_csv_win):
180                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181                 return None
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

37. assistant (2026-06-16T03:13:06.292Z)

Perfect! Now let me look for any existing analysis of fps sensitivity and search for any test/eval files that compare 30 vs 60 fps:

38. user (2026-06-16T03:13:07.388Z)

```
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/audio/e1_probe.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py:        --labels
"golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py:        --fps
59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py:
ap.add_argument("--fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/export_wasb_segments.py:        --labels
"Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/export_wasb_segments.py:
ap.add_argument("--fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:        --fps 59.94
--frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/gemini_refine.py:    ap.add_argument("--
fps", type=float, default=59.94)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/features/rally_seq.py:59.94fps than 30fps) is
exactly the train/serve-skew class this placement prevents.
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/features/rally_seq.py:    speed read ~2x
smaller at 59.94 fps than at 30 fps, confounding cross-footage transfer
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/person_detector.py:
```

```
# Skip frames for performance ? at 30fps, every 3rd frame still
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/validators/resolution_fps.py:
return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate (e.g. 30fps)
requirements."
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/reporting/harvest.py:    # used its internal
fallback (30fps / 1280px) ? surface that as the footgun it is,
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/reporting/harvest.py:            vm["fps"] =
30.0  # matches the runners' fallback (_probe_fps_width -> 30.0 / 1280)
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/tools/export.py:        # EDL is HH:MM:SS:FF
(assume 30fps for the timecode display; real edit rate
.claude/worktrees/agent-a771ff1d85e18f2ed/backend/tools/export.py:            f = int((t -
int(t)) * 30)  # 30 fps display
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibrate_wasb.py:    for i in range(0,
30):          # 0.0-2.9s @10fps: shuttle moving fast
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_person_detector.py:    """A
cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_reporting_harvest.py:    assert
env.video_meta.fps == 59.94
```

39. assistant (2026-06-16T03:13:09.210Z)

Excellent! Let me read that rally_seq.py file that has the fps sensitivity note:

40. user (2026-06-16T03:13:09.816Z)

```
"""Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature schema.

THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
harness (training/) and the serving path import THIS module, so train == serve by
construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.

Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
trained with and the serving loader refuses a mismatch.
"""
```

41. assistant (2026-06-16T03:13:12.023Z)

Great! Let me look at that rally_seq.py file more completely:

42. user (2026-06-16T03:13:12.354Z)

```
1       """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3       THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4       harness (training/) and the serving path import THIS module, so train == serve by
5       construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6       59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8       Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9       MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10      trained with and the serving loader refuses a mismatch.
11      """
12      from __future__ import annotations
13
14      from typing import Dict
15
16      import numpy as np
17      from scipy.ndimage import uniform_filter1d, maximum_filter1d
18
19      # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
20      # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
```

```
    FEATURE_SCHEMA_VERSION = 1

    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]

    # Default windowing ? part of the schema (manifests record the values actually used).
    DEFAULT_WIN_SEC = 0.75
    DEFAULT_STRIDE_SEC = 0.1

    FEATURE_SCHEMA = {
        "id": "rally_seq",
        "version": FEATURE_SCHEMA_VERSION,
        "feat_names": FEAT_NAMES,
        "win_sec": DEFAULT_WIN_SEC,
        "stride_sec": DEFAULT_STRIDE_SEC,
        # Explicit because its absence was a real bug: speed is fraction-of-frame-width
        # PER SECOND (* fps), matching the heuristic windowing brain.
        "fps_semantics": "per-second",
        "occlusion_rule": "speed/crossings not computed across gaps > max(2,
round(0.25*fps)) frames",
        "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
frame_width}",
    }


    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.

        Speed is normalized to fraction-of-frame-width PER SECOND (``* fps``), matching the
        heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
        occlusion gap (mirrors the heuristic resetting on invisibility)."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N)
        x = np.zeros(N)
        y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
                vis[p.frame] = 1.0
                x[p.frame] = float(p.x)
                y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

        speed = np.zeros(N)
        spd_mask = np.zeros(N)
        cross = np.zeros(N)
        max_gap = max(2, round(0.25 * fps))  # frames; a longer gap = occlusion break, not
motion
        prev = None
        for p in pts:
            f = p.frame
            if not (0 <= f < N and vis[f]):
```

```
80                  continue
81              if prev is not None:
82                  df = f - prev
83                  if 0 < df <= max_gap:
84                      speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
85                      spd_mask[f] = 1.0
86                      c, pc = coord[f] - net, coord[prev] - net
87                      if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
88                          cross[f] = 1.0
89              prev = f
90          return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
91
92
93      def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
94                   stride_sec: float = DEFAULT_STRIDE_SEC):
95          """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
96          N = arr["N"]
97          fw = arr["fw"]
98          size = max(3, int(win_sec * fps) * 2 + 1)
99          vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
100
101         n = uniform_filter1d(vis, size, mode="constant")                      # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116
```

43. assistant (2026-06-16T03:13:14.751Z)

Perfect! Now I have all the information I need. Let me create a comprehensive markdown report:

44. user (2026-06-16T03:13:15.572Z)

Chapter marked: "Windowing & signal mapping complete". Continue your current work.

45. assistant (2026-06-16T03:13:57.861Z)

Trajectory-to-Action-Windows: Over-Merge Mechanism & Available Signals

PRIMARY: Windowing Algorithm

`trajectory_to_action_windows()` ? Core Implementation
Location: `backend/pipeline/detectors/tracknet_runner.py:256-323`
Reused by: `backend/pipeline/detectors/wasb_runner.py:199` (staticmethod reference)
Invoked by: `backend/pipeline/segmenters/trajectory_hybrid.py:197-200`

```python
def trajectory_to_action_windows(
    points: List[TrajectoryPoint],
    fps: float,
    frame_width: float,
    chunk_start: float = 0.0,
    velocity_thresh: float = 0.02,
    merge_gap: float = 2.0,
    min_window_duration: float = 2.0,
    chunk_index: Optional[int] = None,
) -> List[Dict[str, Any]]
```

How "Active" Frames Are Determined
Lines 280?294: Frame is marked "active" when:
1. Shuttle is **visible** (`p.visible == True`)
2. **Inter-frame displacement** (Euclidean distance) normalized by `frame_width` and scaled per-second **exceeds `velocity_thresh`**

```python
## Line 290-292: Velocity computation (critical)
dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5  # pixels
velocity = (dist / frame_width) * (fps / dframes)
if velocity > velocity_thresh:
    active.append((p.frame, velocity))
```

Velocity Semantics:
- **Per-frame pixel displacement** normalized by `frame_width` (unitless fraction)
- **Multiplied by `fps / dframes`** ? converts to **per-second** scale
- `dframes = p.frame - prev.frame` (can be > 1 if points are sparse)
- **No FPS-aware compensation beyond the `fps / dframes` factor** ? the problem lies here

Active-Region Merging (Dead-Time Bridge)
Lines 299?321: Contiguous groups of active frames separated by ? `merge_gap` seconds are merged into one window:

```python
max_gap_frames = int(merge_gap * fps)  # Line 299: frame count corresponding to merge_gap seconds
## Line 316: if af[0] - group[-1][0] <= max_gap_frames: merge
```

The Problem ? Why It Over-Merges at 59.94fps:
- At **30fps**: `max_gap_frames = int(2.0 * 30) = 60 frames` ? 2.0 sec of silence
- At **59.94fps**: `max_gap_frames = int(2.0 * 59.94) = 119 frames` ? 2.0 sec of silence

The **frame count threshold doubles**, so at higher fps the temporal tolerance is the **same in seconds** but bridges **twice as many frames**. This causes rallies to merge that should be separate if the dead time happens to fall in the 60?119 frame range.

Window Filtering
Line 323: Drops windows shorter than `min_window_duration`:
```python
return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
```

Candidate-Window Dict Shape
Lines 301?311 (`_finalize` closure):
```python
{
    "start": float,                    # seconds; chunk-relative
    "end": float,                      # seconds; chunk-relative
```

```
    "active_frame_count": int,         # count of frames exceeding velocity_thresh
    "peak_velocity": float,            # max velocity in window
    "mean_velocity": float,            # average velocity in window
    "track_id_count": int,             # always 1 (single shuttle)
    "chunk_index": Optional[int],      # source chunk ID for traceability
}
```

Configuration & Parameter Sources

```
#### Windowing Parameters & Defaults
| Parameter | Default | Config Key | Source |
|-----------|---------|-----------|--------|
| `velocity_thresh` | 0.02 | `indexing.<family>.velocity_threshold` | `trajectory_hybrid.py:160`
|
| `merge_gap` | 2.0 sec | `indexing.dead_time_merge_gap` | `trajectory_hybrid.py:161` |
| `min_window_duration` | 2.0 sec | `indexing.min_action_window_duration` |
`trajectory_hybrid.py:162` |

**Config Schema Location:** `backend/config/models.py:117?154`

- **TrackNetConfig** (lines 33?51): `velocity_threshold: float = 0.02`
- **WasbIndexConfig** (lines 54?81): `velocity_threshold: float = 0.02`
- **FusionConfig** (lines 84?114): `velocity_threshold: float = 0.02` (family-specific override)
- **IndexingConfig** (lines 117?154):
  - `dead_time_merge_gap: float = 2.0`
  - `min_action_window_duration: float = 2.0`

#### NO Maximum Window Duration or Inactivity-Based Split
**Confirmed Absence:** The algorithm has **no upper bound on window duration** and **no
inactivity timeout that splits long windows**. Once two active regions are merged, the resulting
window extends from the first active frame to the last, with no intermediate breaks. This is the
second half of the over-merge problem: a long dead-time bridge cements otherwise-separable
rallies into a single candidate.

---

**SECONDARY: Available Signals for Better Segmentation**

#### 1. Rally Gate (Gate A) ? Net-Crossing Count
**Location:** `backend/pipeline/detectors/rally_gate.py:18?117`

**What it computes:**
- Per-window **shuttle net-crossing count** ? number of sign changes in (shuttle_coord -
net_position)
- Discriminator: real rallies cross the net ?2x; single service feeds cross ~1x
- **Camera-agnostic**: auto-estimates play axis (x or y based on spread) and net position
(median coord)

**Per-frame vs per-window:** Per-window aggregation of frame-level crossings
**Default on/off:** **OFF** (persisted but not gated;
`FusionSegmenter._enrich_candidate_stats:80?85`)
**Persistence:** Stored in `candidate_segments.stats["net_crossings"]` as float
**Gating hook:** `FusionSegmenter._select_for_handoff:154` ? drops windows with `net_crossings <
min_crossings` (default 0 = OFF)

**Key API:**
```python
gate_windows(points, windows: List[Interval], fps,
             min_crossings=2, deadband_frac=0.06, estimate=None)
  -> List[GatedWindow]
```
Returns `GatedWindow(start, end, crossings, visible_points, kept)` per input window.

2. Person Detector ? Motion + Occupancy Cues
```

Location: `backend/pipeline/detectors/person_detector.py:1?299` +
`backend/pipeline/detectors/fusion_features.py:1?215`

What it computes (5 features):
1. **`players_in_court`** (line `fusion_features.py:97?99`): median per-frame in-court person
count
2. **`two_sided_motion`** (line `fusion_features.py:130?155`): fraction of frames with persons
on both net sides
3. **`mean_player_motion`** (line `fusion_features.py:109?127`): mean per-track centre
displacement (normalised by frame dims)
4. **`in_court_ratio`** (line `fusion_features.py:102?106`): fraction of frames with
?min_players in court
5. **`shuttle_player_colocation`** (line `fusion_features.py:168?189`): fraction of visible-
shuttle frames where shuttle sits inside/adjacent to a person box (held vs in-flight)

Per-frame vs per-window: Per-frame detections ? per-window aggregation
Default on/off: **OFF** ? person cue only computes if `indexing.fusion.cue_flags["person"]
== True`
Persistence: Stored in `candidate_segments.stats` with keys matching `PERSON_FEATURE_NAMES`
(line `fusion_features.py:194?197`)
Lazy loading: PersonDetector is instantiated only when person cue is enabled (line
`fusion_hybrid.py:119?120`)

Key APIs:
```python
## Per-chunk feature extraction (yolo_hybrid path):

extract_action_windows_from_chunk(chunk_path, chunk_start, velocity_thresh, merge_gap)
  -> windows with embedded motion stats
```

Per-window enrichment (fusion path):

```
detections_per_frame(video_path, start_frame, end_frame, frame_skip)
  -> {"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width", "frame_height"}

compute_person_features(per_frame, shuttle_points, frame_width, frame_height,
                        court_polygon=None, net=None, min_players=2)
  -> {feature_name: float} for the 5 features above
```

3. Audio Hit Detection ? Racket-Shuttle Contact Events
Location: `backend/pipeline/detectors/audio_features.py:1?34`

What it computes (3 features):
1. **`audio_hit_count`**: number of hit events in window
2. **`audio_hit_rate`**: hits per second
3. **`audio_hit_strength_mean`**: mean strength of detected hits

Per-frame vs per-window: Per-window aggregation of hit events (each has `.t` and
`.strength`)
Default on/off: **OFF** ? expects upstream `HitEvent` list from
`backend/pipeline/audio/hit_detector.py`
Persistence: Columns in `candidate_segments.stats` (line `audio_features.py:15`)

Key API:
```python
compute_audio_features(hits: List[HitEvent], window_start: float, window_end: float)
  -> {"audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"}
```

Note: Hit detection is not integrated into the live pipeline yet (landing note in
`fusion_hybrid.py:92?98`); this is the schema and compute function only.

4. Fusion Features ? Learned Multi-Signal Combination
Location: `backend/pipeline/detectors/fusion_features.py:194?215`

What it computes: The 5 person features (see above) via `compute_person_features()`
Per-frame vs per-window: Per-window (computed once per candidate window)
Default on/off: **OFF** unless `indexing.fusion.cue_flags["person"] == True`
Persistence: Stored directly in `candidate_segments.stats` alongside motion stats

Integration point: `FusionSegmenter._enrich_candidate_stats:76?90` ? always persists net_crossings; conditionally adds person features.

5. Fusion Segmenter Gating Hooks
Location: `backend/pipeline/segmenters/fusion_hybrid.py:76?159`

Two overridable seams:

1. **`_enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)`** (lines 76?90)
   - Adds `net_crossings` (from rally_gate, always)
   - Conditionally adds 5 person features (if `cue_flags["person"]`)
   - Returns dict persisted into `candidate_segments.stats`

2. **`_select_for_handoff(windows, window_stats, idx_cfg)`** (lines 141?159)
   - **Served Gate A**: drops windows with `net_crossings < min_crossings`
   - **Served Gate B**: drops windows with `players_in_court < min_players`
   - Returns indices of kept windows for AI handoff
   - **Persisted candidates remain unchanged** (full superset for offline re-scoring)

Candidate Segments DB Schema
Location: `backend/infrastructure/database.py:97?121` (schema v1), `318?339` (insertion)

```sql
CREATE TABLE candidate_segments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    video_id TEXT NOT NULL,
    detector TEXT NOT NULL,
    chunk_index INTEGER,
    start_time REAL NOT NULL,
    end_time REAL NOT NULL,
    duration REAL NOT NULL,
    confidence REAL,
    stats TEXT,                    -- JSON: motion + net_crossings + person features
    detection_params TEXT,         -- JSON: {velocity_thresh, merge_gap,
min_action_window_duration, ...}
    confirmed_segment_id INTEGER,
    human_label TEXT,
    notes TEXT,
    created_at TIMESTAMP,
    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE SET NULL
)
```

`stats` JSON keys (persisted per candidate):
- Motion (always): `peak_velocity`, `mean_velocity`, `active_frame_count`, `track_id_count`
- Gate A (when fusion): `net_crossings`
- Person features (when fusion + person cue ON): `players_in_court`, `two_sided_motion`, `mean_player_motion`, `in_court_ratio`, `shuttle_player_colocation`
- Audio features (when enabled): `audio_hit_count`, `audio_hit_rate`, `audio_hit_strength_mean`

Root Cause: Why Over-Merge Worsens at 59.94fps

The FPS-Sensitivity Bug (documented in `backend/features/rally_seq.py:1?11`):

At 59.94fps the **window-merging algorithm is unchanged**, but the **frame count threshold doubles**:
- Velocity computation (line `tracknet_runner.py:291`) multiplies by `fps / dframes`, so a given physical shuttle speed reads **similar** across framerates IF the points are densely sampled.
- **BUT** the merge threshold `max_gap_frames = int(merge_gap * fps)` scales linearly with fps.
- A 2-second dead time is always 2 seconds in wall time, but at 59.94fps that's ~120 frames vs ~60 at 30fps.
- Two separate rallies separated by ~1.5s dead time (67?90 frames) stay separated at 30fps but merge at 59.94fps.

Secondary over-merge mechanism:
- High-fps trajectories sample more densely, so small jitter/oscillations at the net appear as more "active frames" within the merge window.
- Without an inactivity-timeout split, this extends window duration unchecked.

Concrete example from eval code (`backend/eval/calibrate_local.py`, `calibrate_wasb.py`, `eval_partial_labels.py`):
- All default to `--fps 59.94` in their docstrings
- Calibration grids search `(velocity_thresh, merge_gap, min_window_duration)` but **velocity_thresh is the only fps-normalized parameter**; merge_gap is always seconds.

Summary Map: Segmentation Signals Ready for Better Design

Signal	Location	Scope	Default	Gating	Persistence
Velocity (heuristic)	`tracknet_runner.py:256?323`	Per-frame ? per-window	0.02	None	`stats.peak_velocity, mean_velocity`
Net Crossings (Gate A)	`rally_gate.py:91?110`	Per-frame ? count	0	`fusion_hybrid:154`	`stats.net_crossings`
Person Motion (Gate B)	`fusion_features.py:97?189`	Per-frame detections ? 5 features	OFF	`fusion_hybrid:156`	`stats.{players_in_court, two_sided_motion, ?}`
Audio Hits	`audio_features.py:18?34`	Per-window	OFF	None	`stats.{audio_hit_count, ?}`

To improve over-merge: Combine Gate A (net crossings ?2 for real rallies) + an **inactivity-timeout split** (max window duration or silence-based break detection) + optional person occupancy (two-sided motion) as a confidence booster.